# Calving Iceberg: A Same-Message Context-Splitting Group-Key Recovery Attack

Martin Feussner
Selmer Center, University of Bergen
martin.feussner@uib.no

18 October 2026

This audit used the OpenAI model Daybreak Blue at the ultra reasoning level. OpenAI Codex generated and selected this paper's title and deliberately chose "Calving Iceberg" as scheme-related wordplay: iceberg calving splits one ice mass, while the attack splits one Iceberg session across several outer contexts. OpenAI Codex performed AI-assisted target reconstruction, attack-history searching, cryptanalysis, implementation, experimentation, review coordination, and technical writing. An independently written fresh-context AI checker reimplemented the secp256k1, BIP327, BIP340, sharing, and extraction logic and returned PASS with negative controls. The exact pinned-source C diagnostic was also rerun and returned PASS. The formal-game, claim-boundary, and prior-art objections were reviewed separately before publication. A separately tasked AI adversarial reviewer attempted to disprove the claim by checking the oracle schedule, attacker-visible inputs, source and profile pins, verifier path, freshness, scope, and attribution, and returned PASS with no blocking correction. Independent human verification was not performed; it is optional and is not a publication requirement.

**Abstract**

Iceberg, proposed in IACR ePrint 2026/1757, lets a threshold group occupy one signer slot in an otherwise ordinary two-party MuSig2 execution. Its deterministic group nonce pair is derived from a session identifier. The paper's TS-MS-EUF-CMA game prevents two messages from being signed under one identifier, but it consumes state separately for each honest member and does not bind the adversary-chosen outer MuSig2 context across those members.

We give a practical full-parameter attack on the paper's concrete $t = 2, n = 4$ secp256k1 profile. Corrupt one member. Query the other three members once each under the same session identifier and the same message, while giving each a different cosigner nonce. Combining each honest response with the corrupt member's local response yields three complete group partial signatures of the form

$$S_j = \varepsilon_j r_1 + \varepsilon_j b_1 b_{0,j} r_2 + e_j a \gamma x.$$

The coefficients are public and three ordinary contexts give a full-rank $3 \times 3$ system. Solving it recovers the two fixed nonce scalars and the group signing key $x$. The recovered key, together with the adversary's outer cosigner key, produces a standard 64-byte BIP340 signature on a never-queried message with no honest group participation.

An exact-source C diagnostic pinned to artifact commit `b2f87cb047467dd78852bc32722a5228a7167f5e` recovered the key and verified the fresh signature in 0.01 seconds using 2,560 KiB peak resident memory. An independently written standard-library Python implementation reproduced the result in 0.95 seconds wall time (0.92 seconds user time) using 15,360 KiB. Both use the ordinary BIP340 verifier, and negative controls reject repeated contexts, two equations, omitted nonce parity, and raw unaggregated responses. The three-response MuSig2 solve, same-message context manipulation, and replicated-sharing split ingredients are prior public knowledge; we claim only their concrete application to Iceberg's formal game. No previous public report of this complete attack against Iceberg was located in searches through 18 October 2026.

# 1 Introduction

Iceberg is a nested threshold multisignature intended to replace one side of a Lightning channel's two-party MuSig2 signer with a threshold group while leaving the counterparty and the on-chain BIP340 verifier unchanged [1, 3, 4]. A key requirement is that the group commit to its nonce before the transaction message and final signing quorum are known. To make that possible, Iceberg derives two group nonce scalars deterministically from replicated secret shares and a session identifier.

Deterministic MuSig nonces require more than message equality. A malicious cosigner can keep the message fixed and change its nonce, thereby changing the aggregate nonce coefficient and Schnorr challenge. This general phenomenon is known from deterministic multisignature analyses, and the direct recovery of a BIP327 key from three responses using one fixed two-scalar nonce pair is also public [7, 5]. Iceberg attempts to prevent dangerous reuse by requiring group agreement on one message per session identifier.

The formal game enforces exactly that message condition, but no corresponding condition on the complete second-round input. Each honest member has its own ready/done state. Consequently, three different honest members can each answer once for one common message and identifier while seeing three different outer cosigner nonces. Threshold interpolation turns these three distinct member responses into three complete responses for the same group-level key and nonce unknowns. The published three-response linear solve then recovers the group key.

**Contributions.** We (i) give the exact game-admissible query schedule; (ii) derive the complete BIP327 equation including both nonce and aggregate-key parity signs; (iii) execute key recovery and a fresh BIP340 forgery against the pinned full-parameter source; (iv) reproduce the result independently with a separate secp256k1 implementation and negative controls; and (v) identify the precise stronger assumption used by the proof but absent from Figure 3.

**Originality boundary.** We do not claim a new nonce-reuse technique. Kohen gives the three-response BIP327 linear system and discusses replicated-sharing quorum splits [5]; MuSig-DN discusses same-message context manipulation for a deterministic one-nonce multisignature [7]; and the public Iceberg source itself contains a commented observation that changing the counterparty's nonce can change the signature hash without changing the message. Our result is the concrete synthesis for Iceberg: its message-only game binding and member-local state supply three legal complete group responses in the claimed 2-of-4 profile.

# 2 Target and exact profile

The target is the initial/current version of *Enabling Threshold Custody for the Lightning Network with Nested Threshold Multi-Signatures*, IACR ePrint 2026/1757, received 21 August and approved 22 August 2026 [1]. The retained 36-page PDF has SHA-256 digest

`6a3bd9d2079551000ea4d69e18463f0975e85decbb5a37f0efff30e1c2508403.`

The independently downloaded arXiv v1 source archive has SHA-256 digest

`984b33ba56392a9e5aef5b64a0bd58c4946fe458dae3afd26452c8dfd1198ef5.`

The retained profile is

`ICEBERG-SECP256K1-T2-N4-MU3-NU2-DEPTH2-`
`BIP327-BIP340-SID32-ARTIFACT-B2F87CB.`

It uses the prime-order secp256k1 group, threshold $t = 2$, four members, one static corruption, the paper's $\mu = 2t - 1 = 3$ live-member policy, two MuSig2 nonce components, 32-byte messages and identifiers, one fixed attacker-owned outer cosigner, the group key first in the two-key list, and no BIP341 tweak. This is the paper's smallest and benchmarked deployment profile, not a reduced security parameter.

The author artifact is pinned at commit `b2f87cb047467dd78852bc32722a5228a7167f5e` [2]. The C diagnostic includes its vendored secp256k1 implementation directly and calls the native Iceberg share generation, public-share aggregation, nonce generation, nonce aggregation, partial-signature generation and verification, partial aggregation, BIP327 session, and BIP340 verification paths. No implementation behavior is needed to create the weakness: the query schedule is expressly permitted by the paper's game and the equations are those of the construction.

# 3 Iceberg signing equations

Let $q$ be the secp256k1 group order and write scalar arithmetic in $\mathbb{Z}_q$. The threshold group holds signing scalar $x$ and public key $X = xG$. For a session identifier $sid$, Iceberg deterministically derives two shared nonce scalars $r_1, r_2$ and public points $R_1 = r_1G$ and $R_2 = r_2G$. Its inner coefficient is

$$b_1 = H_{\text{Iceberg/noncecoef}}(R_1, R_2, X),$$

using the source's extended point encodings. The group exports the ordinary MuSig2 public nonce $(R_1, b_1R_2)$.

Fix an outer key list $[X, X_B]$, where the adversary owns $x_B$ and $X_B = x_BG$. BIP327 key aggregation assigns public coefficient $a$ to $X$ and forms the aggregate key $\widetilde{X} = aX + X_B$. Let $\gamma \in \{1, -1\}$ be the source's aggregate-key parity correction. In outer context $j$, the adversary supplies a fresh valid cosigner nonce pair. Aggregating it with the fixed group nonce yields the public outer nonce coefficient $b_{0,j}$, final nonce parity correction $\varepsilon_j \in \{1, -1\}$, and BIP340 challenge $e_j$. Every one of these values is public.

If member $i$ holds Shamir-form shares $x_i, r_{1,i}, r_{2,i}$, its second-round response in context $j$ is

$$s_{i,j} = \varepsilon_j r_{1,i} + \varepsilon_j b_1 b_{0,j} r_{2,i} + e_j a\gamma x_i \pmod q. \tag{1}$$

Lagrange interpolation of any $t$ valid member responses therefore produces the group's ordinary MuSig2 partial scalar

$$S_j = \varepsilon_j r_1 + \varepsilon_j b_1 b_{0,j} r_2 + e_j a\gamma x \pmod q. \tag{2}$$

For $t = 2$, corrupt member 1 and honest member $i$ have interpolation weights

$$\lambda_1 = \frac{i}{i-1}, \qquad \lambda_i = -\frac{1}{i-1},$$

so the adversary computes $S_j = \lambda_1 s_{1,j} + \lambda_i s_{i,j}$. It knows $s_{1,j}$ from the corrupt member's shares and obtains $s_{i,j}$ from one honest oracle call.

# 4 Same-message context-splitting attack

## 4.1 Legal oracle schedule

Figure 3 defines a global message map $M[sid]$ and a separate state $SID[i][sid]$ for each member. The pre-round oracle changes one member's state from unset to ready. The signing oracle checks only that the selected member is ready and

$$M[sid] \in \{\perp, m\},$$

then calls SignRound on the adversary-supplied aggregate pre-round message and outer context, marks only that member done, and records the member in $Q[m, sid]$. The paper explicitly describes the outer context as including the group position, cosigner public keys, and outer aggregate first-round message, and says that the adversary chooses it.

The following schedule is therefore admitted for one corrupt member:

1. choose one session identifier $s$ and query PreRound$(s, i)$ once for each honest member $i \in \{2, 3, 4\}$;
2. aggregate those three valid pre-round contributions into the fixed group public nonce;
3. fix one message $m$, one outer key list $[X, X_B]$, and one attacker cosigner key $x_B$;
4. choose three distinct valid cosigner nonce pairs, producing contexts $ctx_2, ctx_3, ctx_4$;
5. query honest member $i$ exactly once on $(s, i, m, out, ctx_i)$.

Every query uses the same $s$ and $m$, so the global message check succeeds. Every member answers once, so its local state check succeeds. Only the adversary-controlled cosigner nonce changes.

## 4.2 Key extraction

Combine each honest response with the locally generated corrupt response to obtain $S_2, S_3, S_4$. Equation (2) gives

$$\begin{pmatrix} S_2 \\ S_3 \\ S_4 \end{pmatrix} = \begin{pmatrix} \varepsilon_2 & \varepsilon_2 b_1 b_{0,2} & e_2 a\gamma \\ \varepsilon_3 & \varepsilon_3 b_1 b_{0,3} & e_3 a\gamma \\ \varepsilon_4 & \varepsilon_4 b_1 b_{0,4} & e_4 a\gamma \end{pmatrix} \begin{pmatrix} r_1 \\ r_2 \\ x \end{pmatrix} \pmod q.$$

The adversary knows the matrix and right-hand side. Distinct outer nonces change $b_{0,j}$ and $e_j$; ordinary choices give rank three except for a negligible degeneracy that is detected before submitting the queries. The deterministic reproduction selected candidates 0, 1, and 5. Their parity signs were $(-1, -1, +1)$ and their matrix had rank three. Gaussian elimination recovered $(r_1, r_2, x)$, and $xG$ exactly matched the public group key.

### 4.3 Fresh-message forgery

The adversary now knows both secret halves of the outer two-party signing relation: recovered group scalar $x$ and its own cosigner scalar $x_B$. It runs ordinary MuSig2 locally on a new message $m^* \neq m$ and outputs the resulting 64-byte BIP340 signature under $\mathsf{KeyAgg}([X, X_B])$. The standard BIP340 verifier accepts; changing one byte of $m^*$ makes it reject.

No honest member signs $m^*$. For every identifier, $Q[m^*, sid]$ is empty. With $t = 2$ and one corrupt member, Figure 3's win condition is

$$|Q[m^*, sid]| = 0 < t - |C| = 1.$$

The result is therefore both secret-key recovery with fresh signing and an accepted fresh-message forgery under the paper's claimed security game.

**Theorem 1.** *In the Iceberg TS-MS-EUF-CMA game instantiated at $t = 2, n = 4$, a static adversary corrupting one member recovers the group signing key after three honest second-round queries on one message and one session identifier, and then forges a fresh BIP340 signature with no honest participation, except with the negligible probability that no tested context triple has full rank.*

*Proof.* The three queries are legal by the member-local state and message-only global check above. Each corrupt-plus-honest pair interpolates Equation (1) to Equation (2). A full-rank coefficient matrix uniquely reveals $r_1, r_2, x$. Equality $xG = X$ publicly validates the recovered key. Knowledge of $x$ and the controlled cosigner key permits ordinary BIP327 signing on a fresh message, whose final signature is accepted by BIP340. The forged message has no honest signing response, so it satisfies the game's freshness inequality. □

## 5 Proof gap and design status

The first reduction says that the consensus assertion permits only one message per identifier and concludes that one underlying signing-oracle call per session suffices. The main reduction later assumes the stronger statement that state agreement makes it impossible to query the same identifier on "two different inputs to the second round." Figure 3 enforces equality only of $m$; its other second-round inputs remain adversary-controlled. The three queries above are therefore distinct second-round inputs but the same allowed message.

This is a design-level attack on the formal construction, not an accidental parser or implementation defect. It uses valid group elements, a claimed parameter set, fewer than $t$ corruptions, authenticated member identities, one response per honest member, one message per identifier, and the ordinary outer verifier. It does not require rollback, repeated use by one member, malformed encodings, weak keys, a hash collision, or an implementation race. The pinned source execution is conformance evidence for the coefficients and verifier endpoint, rather than the source of the weakness.

The attack assumes the adversarial outer signer and network powers already granted by the TS-MS-EUF-CMA game. A deployment that authenticates and reaches group-wide agreement on the entire second-round tuple before any member answers would block this schedule, but that requirement is stronger than the message agreement encoded in Figure 3 and used in the stated security claim.

## 6 Experimental validation

### 6.1 Pinned-source diagnostic

The exact-source C diagnostic used artifact commit `b2f87cb047467dd78852bc32722a5228a7167f5e`. The retained attack source has SHA-256

`e9787f333a1cd09411629eae9d1751de5c21e29f5b61e8d13102c708ec624e01.`

It was compiled with GCC 13.3.0 at C99 and optimization level `-O2`. The program generated a trusted-dealer $t = 2, n = 4$ instance, used honest members 2, 3, and 4 for the three pre-round contributions, generated one response per honest member in three contexts, verified every member response, completed and verified all three queried-message outer signatures, solved the system, regenerated the group public key, and verified a fresh-message signature.

The attack executable took 0.01 seconds, used 2,560 KiB maximum resident memory, and used one worker. The local rerun reproduced every recorded scalar and result flag. The only byte difference in captured standard output was an extra trailing blank line in the sender's archived transcript.

### 6.2 Independent reproduction

A separately written 487-line Python checker uses only the standard library. It implements secp256k1 point arithmetic, tagged hashes, Iceberg dealer and VPSS derivation, Lagrange interpolation, BIP327 key and nonce coefficients, BIP340 challenge, signing, and verification. Its adversary receives only the public transcript, the corrupt member's shares, three honest responses, and the controlled cosigner secret. Fixture-wide secrets are used only for post-recovery assertions.

The coordinator replay of the checker returned PASS in 0.95 seconds wall time (0.92 seconds user time) with 15,360 KiB maximum resident memory on one process and worker. Its JSON output matched the supplied independent result byte for byte. It confirmed the following negative controls:

- two equations have rank two and do not determine three unknowns;
- repeating one outer context leaves the three-row matrix at rank two;
- omitting final-nonce parity makes the true solution violate the equations; and
- treating raw honest responses as complete group partials produces a scalar whose public point is not the group public key.

These are deterministic full-instance reproductions, not random-key statistical trials. The algebra is generic, while the reported run establishes one exact full-parameter key and transcript. The public package includes the independent Python replay and the exact C diagnostic source.

## 7 Prior work and attack-history search

Kohen's March 2026 note gives the direct three-response MuSig2 solve for the two fixed nonce scalars and signing key, and separately describes the danger of splitting replicated nonce material among different quorums [5]. Ruffing and Kohen also discuss the different limited-reuse setting in which each BIP327 nonce pair is exposed only twice and many pairs are combined with a generalized-birthday method [6]. The present attack needs one nonce pair and three complete responses.

MuSig-DN describes same-message context manipulation in a deterministic multisignature: a malicious cosigner changes its nonce and hence the challenge while honest nonce material repeats [7]. Its displayed extraction has one effective nonce scalar and does not use different threshold members. The Iceberg authors' public explanation discusses a replicated-sharing split across different messages and presents message agreement as the defense [8]. The attack here keeps the message fixed and varies only the outer context that Figure 3 leaves unbound.

Searches through 18 October 2026 covered the exact paper title and identifier, Iceberg and nested-MuSig aliases, authors, the artifact repository, issues, pull requests, releases, scholarly and general search results, same-message context attacks, MuSig2 nonce reuse, related Nested MuSig2 work, and the public author material. No previous public report of the complete Iceberg-specific attack was located. This is a dated, bounded statement; it does not exclude private, deleted, unindexed, or later work, and it is not a claim that the underlying techniques are new.

## 8 Repair direction and limitations

A repair must bind every honest member under one identifier to one complete second-round computation, including the group aggregate pre-round message, group position, outer key list, outer aggregate nonce, message, and any tweak state. Checking the message alone is insufficient. Possible designs include a

group-agreed digest of the whole context or a revised state machine that stores that digest before returning any share. Such a change must coexist with the paper's requirement that the group nonce be fixed before the message exists, and it needs a new proof; we do not claim a validated drop-in repair.

The result is limited as follows:

- the executed source-backed endpoint is the claimed $t = 2, n = 4$, $\mu = 3$, $\nu = 2$ secp256k1 profile;
- the attack targets unforgeability in the paper's formal game and does not evaluate Lightning consensus, availability, privacy, or channel state safety beyond the stated interface;
- the source artifact is a benchmark prototype with a trusted dealer; the attack does not require or claim a DKG or production integration;
- the prior-art conclusion is bounded to public material located by the search date; and
- no novelty is claimed for the linear solve, deterministic-nonce context manipulation, or replicated-sharing quorum-split ideas.

## Acknowledgements

The computations were performed on the Norwegian Research and Education Cloud (NREC), using resources provided by the University of Bergen and the University of Oslo.

## References

[1] P. Gerhart, N. Kohen, J. Posner, and M. Furszyfer. Enabling Threshold Custody for the Lightning Network with Nested Threshold Multi-Signatures. IACR Cryptology ePrint Archive, Report 2026/1757, 2026. https://eprint.iacr.org/2026/1757.

[2] M. Furszyfer et al. Iceberg benchmark artifact, commit b2f87cb047467dd78852bc32722a5228a7167f5e. https://github.com/furszy/benchmark-iceberg/commit/b2f87cb047467dd78852bc32722a5228a7167f5e.

[3] J. Nick, T. Ruffing, and E. J. Jin. BIP 327: MuSig2 for BIP340-compatible Multi-Signatures, 2022. https://github.com/bitcoin/bips/blob/master/bip-0327.mediawiki.

[4] J. Nick, T. Ruffing, and P. Wuille. BIP 340: Schnorr Signatures for secp256k1, 2020. https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki.

[5] N. Kohen. A Practical Attack Against Limited MuSig2 Nonce Reuse, 19 March 2026. https://nkohen.github.io/blog/musig2-nonce-reuse/.

[6] T. Ruffing. How many nonce reuse before exposing your MuSig2 private key?, Delving Bitcoin, 7 March 2024. https://delvingbitcoin.org/t/how-many-nonce-reuse-before-exposing-your-musig2-private-key/217/4.

[7] J. Nick, T. Ruffing, Y. Seurin, and P. Wuille. MuSig-DN: Schnorr Multi-Signatures with Verifiably Deterministic Nonces. IACR Cryptology ePrint Archive, Report 2020/1057, 2020. https://eprint.iacr.org/2020/1057.

[8] N. Kohen. Iceberg: Nested Threshold Signatures for Lightning, 28 August 2026. https://nkohen.github.io/blog/iceberg/.